Quantum Random Number Generation for Securing IoT Devices: A Noise-Aware Simulation Study with NIST SP 800-22 Evaluation

Urvashi, Mehul, Pritam
Noida Institute of Engineering and Technology
Greater Noida, India

***Abstract*—Internet of Things (IoT) devices depend on random numbers for encryption keys, nonces and session tokens, yet many of them generate these values with deterministic pseudo-random number generators (PRNGs) seeded from low-entropy sources at boot time. This paper studies Quantum Random Number Generation (QRNG) as a source of key material for IoT devices. An eight-qubit Hadamard-and-measure circuit is implemented in Qiskit and run on (i) an ideal simulator and (ii) a noise model built from the published calibration of the 127-qubit *ibm_brisbane* superconducting processor, including asymmetric readout errors. Raw noisy output is post-processed with a per-qubit Von Neumann corrector and with a SHA-256 extractor, and all streams are compared with three classical baselines: MT19937, the operating-system CSPRNG, and a weakly seeded linear congruential generator (LCG) of the kind still found in embedded firmware. Each source is evaluated on 10^7 bits using eight NIST SP 800-22 tests (100×10^5-bit sequences), entropy and serial-correlation metrics, a machine-learning next-bit predictor, and an IoT key-generation and seed-recovery experiment. Raw noisy QRNG output had an overall bias of 0.0218 and failed all eight tests, with one qubit reaching $P(1) = 0.6835$. After Von Neumann or SHA-256 post-processing it passed all eight tests, and next-bit prediction accuracy fell to chance (50.06% and 49.81%, $\pm$0.40%). The weak LCG reached perfect 100/100 pass proportions on five of the eight tests (failing only on P-value uniformity), yet its next bit was predicted with 95.03% accuracy, it produced only 16 distinct keys out of 10,000, and its seed was recovered for 200/200 simulated devices. The results show that QRNG output needs device-aware post-processing before use and that passing statistical tests alone does not establish that a generator is safe for key generation.**

***Index Terms*—quantum random number generator, Internet of Things, Qiskit, NIST SP 800-22, randomness extraction, Von Neumann corrector, key generation, NISQ noise**

I. Introduction

Random numbers underpin almost every security function on an IoT device: symmetric session keys, elliptic-curve private keys, nonces for (D)TLS and OSCORE, and random back-off in medium-access protocols [1]. If an adversary can predict these values, encryption offers no protection. IoT devices make this hard to get right. They are often headless, have little hardware entropy, and generate their keys automatically on first boot. In an Internet-wide survey of TLS and SSH hosts, Heninger *et al.* were able to compute private keys for a measurable fraction of hosts because of insufficient entropy at key-generation time, and found that the affected hosts were overwhelmingly headless and embedded network devices [2].

A PRNG is a deterministic algorithm: given the seed, its whole output is fixed. Quantum mechanics offers a different source. Measuring a qubit prepared in an equal superposition gives an outcome that, by the Born rule, cannot be predicted even in principle. Gate-based cloud quantum computers can also act as QRNGs, but their qubits are noisy, and earlier work found that some qubits are noticeably more biased than others [3], [4].

This paper takes a deliberately simple, reproducible approach to the question *"Can a gate-based QRNG supply key material for IoT devices, and what processing does it need?"* The contributions are:

1) A complete Qiskit QRNG pipeline (circuit, calibrated noise, post-processing, key formatting) that runs on a laptop and can be pointed at real IBM Quantum hardware without changes.
2) A noise model that uses calibrated *asymmetric* readout probabilities. We show that the default symmetric readout model in the simulator hides the measurement bias entirely, because a symmetric bit-flip leaves a 50/50 distribution unchanged.
3) A side-by-side evaluation of seven generators using eight NIST SP 800-22 tests, entropy metrics, a learned next-bit predictor (used here as the "accuracy" measure of predictability), and an IoT key-generation and seed-recovery experiment.
4) Evidence that a weakly seeded embedded PRNG can show perfect pass proportions on most of the statistical tests and still be completely broken in practice.

II. Related Work

QRNGs on cloud quantum computers. Tamura and Shikano generated random numbers on IBM's 20-qubit cloud processor, applied NIST SP 800-22, and found that bias and stability differed from qubit to qubit. They proposed the tests as a simple diagnostic of qubit condition [3]. Li *et al.* implemented a source-independent QRNG protocol on IBM's cloud processors, bounding the superposition-preparation error so that randomness is certified even with readout errors [4].

Randomness for the IoT. Kietzmann *et al.* analyse random-number generation from the perspective of an IoT operating system, evaluate hardware sources and software generators on constrained devices, and give recommendations [1]. Heninger

TABLE I
SUMMARY OF EARLIER WORK AND THE GAP ADDRESSED HERE

Work	What they did	Key finding	What was missing
Heninger *et al.* (2012) [2]	Internet-wide scan of TLS/SSH keys	Weak randomness let private keys be computed, mostly on embedded devices	No quantum solution
Kietzmann *et al.* (2021) [1]	Studied random generators in IoT operating systems	Guidelines for IoT random generators	Classical generators only
Tamura & Shikano (2021) [3]	QRNG on IBM 20-qubit cloud computer with NIST tests	Some qubits more biased than others	Not linked to IoT keys
Li *et al.* (2021) [4]	Source-independent QRNG on IBM cloud computers	Randomness certified even with readout errors	No IoT comparison or attack tests
Von Neumann (1951) [5]	Pair method for removing bias	Makes biased independent bits fair	A method, not an application
NIST SP 800-22 (2010) [6]	Official randomness test suite	Tests alone cannot replace security analysis	A standard, not an application
This work	**QRNG with real IBM errors, cleaned, compared with weak IoT generators**	**Cleaned bits pass 8/8 tests and are unpredictable; weak IoT LCG 95.03% predictable, 200/200 seeds recovered**	**Fills the gap**

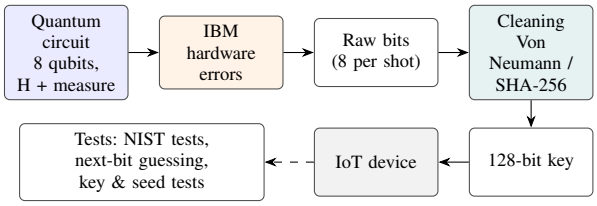


Fig. 1. Proposed QRNG-for-IoT pipeline.

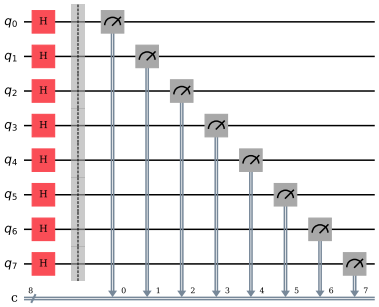


Fig. 2. Eight-qubit QRNG circuit: one Hadamard per qubit, then measurement.

et al. documented the real-world consequence of low boot-time entropy [2].

Post-processing. Von Neumann's classic corrector removes bias from independent but biased bits [5]. NIST SP 800-22 provides the statistical test suite used here [6].

Gap. Earlier gate-based QRNG studies focus on the physics or on qubit diagnostics. This work connects a gate-based QRNG to the IoT key-generation use case and compares it with the weak embedded PRNGs it would replace. It reports security-relevant measures (next-bit predictability, key uniqueness and seed recovery) alongside the usual statistical tests. Table I summarises the earlier work and where ours fits.

III. Background

A. Quantum randomness from one gate

A qubit starts in $|0\rangle$. The Hadamard gate gives

$$H|0\rangle = \tfrac{1}{\sqrt{2}}\left(|0\rangle + |1\rangle\right), \tag{1}$$

and by the Born rule a measurement in the computational basis yields 0 or 1, each with probability $|1/\sqrt{2}|^2 = 1/2$. With n qubits in parallel, one circuit execution (a "shot") yields n bits.

B. Noise and bias

On real hardware the measurement is imperfect. Let $e_{10} = P(\text{read } 1 \,|\, 0)$ and $e_{01} = P(\text{read } 0 \,|\, 1)$. For an ideal superposition the observed probability is

$$P(1) = \tfrac{1}{2}\left(1 - e_{01}\right) + \tfrac{1}{2}\, e_{10} = \tfrac{1}{2} + \tfrac{1}{2}\left(e_{10} - e_{01}\right). \tag{2}$$

The bias therefore depends only on the *asymmetry* $e_{10} - e_{01}$. A symmetric error ($e_{10} = e_{01}$) leaves the output perfectly balanced, which is why the readout model has to be asymmetric.

C. Min-entropy and extraction

The worst-case unpredictability of a bit is its min-entropy, $H_\infty = -\log_2 \max(p_0, p_1)$. Two extractors are used. The *Von Neumann corrector* reads bit pairs, outputs the first bit of 01 or 10, and discards 00 and 11. For independent bits with any fixed bias, the output is exactly unbiased, at a yield of $p(1-p)$ bits per input bit [5]. The *SHA-256 extractor* compresses each 512-bit raw block to 256 output bits, a compression ratio of 0.5 that we keep below the lowest per-qubit min-entropy measured (Section V-A).

IV. Methodology

A. System architecture

Fig. 1 shows the pipeline. An IoT device (for example an ESP32 or Raspberry Pi) asks a QRNG service for key material. The service runs the circuit, post-processes the raw bits, and returns a 128-bit key in a format the device's crypto library can use.

B. Quantum circuit

The circuit (Fig. 2) applies a Hadamard gate to each of eight qubits and measures them. It is transpiled to the backend's native basis ($\sqrt{X}$, R_z), with an initial layout pinned to physical qubits 0–7. Bits are read in qubit order (q0 to q7) for each shot.

C. Noise model

For the noisy runs we use the calibration snapshot of the *FakeBrisbane* backend, which reproduces IBM's 127-qubit *ibm_brisbane* Eagle processor. Gate and thermal-relaxation errors come from `NoiseModel.from_backend`. When we first ran the default backend model, the output looked

perfectly unbiased. On inspection, the simulator's default readout error for this backend is symmetric (for q5, $P(1|0) = P(0|1) = 0.1855$), which by Eq. (2) cannot create bias. We therefore replaced it with each qubit's calibrated asymmetric assignment probabilities (`prob_meas1_prep0`, `prob_meas0_prep1`), listed in Table II.

D. Generators compared

Seven sources, each producing 10^7 bits:

- **Ideal QRNG**: noiseless Aer simulator.
- **Noisy QRNG (raw)**: calibrated noise, no post-processing.
- **Noisy + Von Neumann**: corrector applied *per qubit* (pairs taken from consecutive shots of the same qubit), so pairs never mix qubits with different biases.
- **Noisy + SHA-256**: $512 \to 256$-bit hashing of raw blocks.
- **MT19937**: Python's `random` module.
- **OS CSPRNG**: `os.urandom` (Linux kernel).
- **Weak IoT LCG**: $s_{k+1} = (1103515245\, s_k + 12345) \bmod 2^{31}$, emitting the low byte of each state. This mirrors the common firmware pattern `rand() & 0xFF` seeded from a boot timer.

E. Evaluation protocol

NIST SP 800-22 [6]: eight tests (Frequency, Block Frequency M=128, Runs, Longest Run M=128, Cumulative Sums, Approximate Entropy m=8, Serial m=8, DFT) on 100 sequences of 10^5 bits, significance $\alpha = 0.01$. Following NIST, a test passes if at least 96/100 sequences pass and the uniformity P-value of the 100 P-values is $\geq 10^{-4}$.

Statistical metrics: bias $|P(1) - 0.5|$, per-bit min-entropy, a byte-level χ^2 uniformity test, and lag-1 autocorrelation.

Next-bit predictability ("accuracy"): a gradient-boosted tree classifier is trained to predict bit b_t from the previous 16 bits, using 140,000 training and 60,000 test windows. For a good generator the test accuracy should be indistinguishable from 50% (95% CI $\pm$0.40%).

IoT key experiment: 10,000 consecutive 128-bit keys per source. We count distinct keys, measure pairwise Hamming distance (ideal: 64 ± 5.66), and record the maximum per-position bias.

Seed-recovery attack: 200 simulated devices each seed the LCG with a random 16-bit value and generate two keys. The attacker sees key 1, searches all 2^{16} seeds, and checks whether key 2 is predicted correctly.

Setup: Python 3, Qiskit 2.6.1, Qiskit Aer 0.18.2, qiskit-ibm-runtime fake provider, NumPy/SciPy, scikit-learn 1.9.1. The NIST tests were implemented following the formulas in [6].

V. Results

A. Per-qubit bias under realistic noise

Table II and Fig. 3 show the measured output probability of each qubit. In agreement with Eq. (2), the bias follows the readout asymmetry. Qubit 5 ($e_{10} = 0.369$, $e_{01} = 0.002$) gives $P(1) = 0.6835$ and a min-entropy of only 0.549 bits per bit. Even the best qubits (q0, q1) fail the per-qubit frequency test in some sequences, because at 10^5 bits a 0.25% bias is detectable. After the per-qubit Von Neumann corrector, every qubit sits at 0.500 ± 0.0006 and passes all its frequency tests. The overall Von Neumann yield was 21.6% of raw bits (limited by q5). The lowest per-qubit min-entropy (0.549) exceeds the 0.5 compression ratio of the SHA-256 extractor.

TABLE II
Per-qubit calibration and measured randomness (noisy runs)

	Readout error		Raw			Von Neumann	
Qubit	e_{10}	e_{01}	$P(1)$	H_∞	Freq. pass	$P(1)$	Freq. pass
q0	0.0254	0.0303	0.4975	0.993	55/63	0.5001	15/15
q1	0.0293	0.0327	0.4981	0.995	59/63	0.4998	15/15
q2	0.0044	0.0171	0.4934	0.981	2/63	0.5000	15/15
q3	0.0459	0.0088	0.5186	0.947	0/63	0.4998	15/15
q4	0.0327	0.0176	0.5074	0.979	1/63	0.4994	15/15
q5	0.3691	0.0020	0.6835	0.549	0/63	0.4995	13/13
q6	0.0098	0.0371	0.4861	0.960	0/63	0.5000	15/15
q7	0.0088	0.0244	0.4918	0.977	0/63	0.5002	15/15

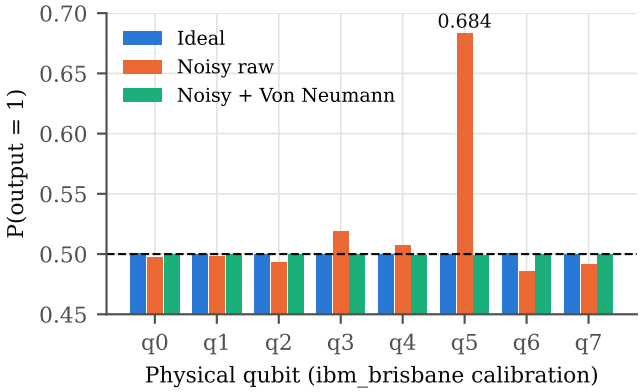


Fig. 3. Probability of output 1 per physical qubit. Calibrated asymmetric readout error biases the raw bits (orange); per-qubit Von Neumann correction restores 0.5 (aqua).

B. NIST SP 800-22 results

Table III and Fig. 4 summarise the eight tests. The ideal QRNG, both post-processed QRNG streams, MT19937 and the OS CSPRNG pass all eight. Raw noisy QRNG fails all eight (overall $P(1) = 0.5218$). The weak LCG reaches 100/100 pass proportions on the Frequency, Block Frequency, Runs, CuSum and Serial tests, but its P-values are not uniformly distributed (uniformity $P < 10^{-4}$). By the NIST criteria it therefore fails these tests as well as Longest Run, ApEn and DFT. On the frequency-style tests alone it looks *too perfect*: it has exactly zero bias because the low byte cycles through all 256 values with period 256.

C. Entropy and predictability

Table IV reports the statistical metrics and next-bit prediction accuracy, and Fig. 5 plots the accuracy. Every generator that passes NIST is predicted at chance level ($50 \pm 0.4\%$). Raw noisy QRNG is predicted with 52.02% accuracy, enough

TABLE III
NIST SP 800-22 RESULTS: SEQUENCES PASSING OUT OF 100 ($\alpha = 0.01$, PASS THRESHOLD 96/100). BOLD = FAIL; † = PROPORTION PASSES BUT P-VALUE UNIFORMITY FAILS ($< 10^{-4}$).

Source	Frequency	Block Freq.	Runs	Longest Run	Cum. Sums	Approx. Entropy	Serial	DFT	Tests passed
Ideal QRNG	100	100	99	99	100	99	100	100	8/8
Noisy QRNG (raw)	**0**	**5**	**0**	**2**	**0**	**0**	**0**	**10**	0/8
Noisy + Von Neumann	100	99	100	98	100	99	99	100	8/8
Noisy + SHA-256	99	100	100	99	99	100	97	99	8/8
MT19937 PRNG	99	100	99	100	100	100	100	99	8/8
OS CSPRNG	99	99	100	100	99	100	97	97	8/8
Weak IoT LCG	**100**†	**100**†	**100**†	**0**	**100**†	**0**	**100**†	**0**	0/8

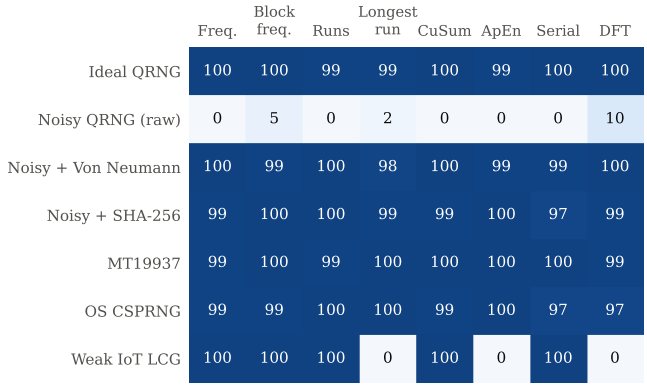


Fig. 4. NIST SP 800-22 pass counts (out of 100 sequences) for each source and test.

TABLE IV
STATISTICAL METRICS AND NEXT-BIT PREDICTION ACCURACY (10^7 BITS PER SOURCE)

Source	Bias	H_∞/bit	Byte χ^2 p	Lag-1 AC	Next-bit acc. (%)
Ideal QRNG	0.00004	0.9999	0.262	-0.0004	50.04
Noisy QRNG (raw)	0.02183	0.9384	0.000	-0.0013	52.02
Noisy + Von Neumann	0.00021	0.9994	0.819	-0.0004	50.06
Noisy + SHA-256	0.00017	0.9995	0.643	+0.0011	49.81
MT19937 PRNG	0.00022	0.9994	0.006	-0.0005	50.21
OS CSPRNG	0.00019	0.9995	0.365	+0.0015	50.36
Weak IoT LCG	0.00000	1.0000	1.000	-0.0000	95.03

for an attacker to gain an advantage because the model learns the q5 bias from the bit context. The weak LCG reveals its weakness here: its next bit is predicted with 95.03% accuracy, even though its bias, min-entropy and byte χ^2 look ideal or better (χ^2 $p = 1.000$ is itself a warning sign of excessive regularity).

D. IoT key generation and seed recovery

Table V reports the key-level results. All QRNG and good classical streams yield 10,000 distinct keys with mean pairwise Hamming distance close to the ideal 64 (for example 63.99 for Noisy + SHA-256). Raw noisy QRNG keys have one bit position biased by 0.19 (the q5 position in every byte). The weak LCG produces only 16 distinct keys in 10,000, because its 256-byte period repeats every 16 keys, and their mean Hamming distance is only 32.04. In the seed-recovery attack,

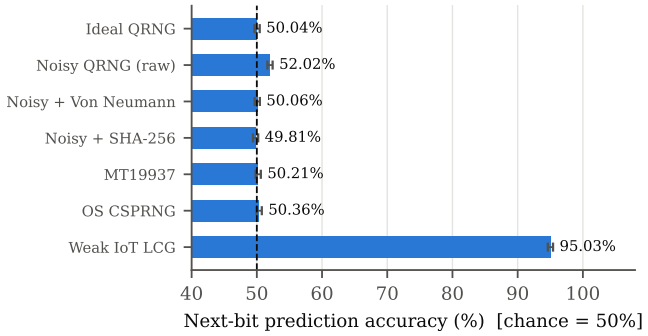


Fig. 5. Accuracy of a learned next-bit predictor (16-bit context, 60,000 test windows; error bars = 95% CI). The dashed line marks chance.

TABLE V
IOT KEY GENERATION (10,000 × 128-BIT KEYS) AND THROUGHPUT

Source	Unique keys	Hamming dist.	Max pos. bias	Mbit/s*
Ideal QRNG	10,000	64.04 ± 5.67	0.010	1.98
Noisy QRNG (raw)	10,000	62.90 ± 5.66	0.191	1.39
Noisy + Von Neumann	10,000	64.10 ± 5.63	0.014	0.30
Noisy + SHA-256	10,000	63.99 ± 5.62	0.014	0.69
MT19937 PRNG	10,000	63.96 ± 5.70	0.016	2069
OS CSPRNG	10,000	63.90 ± 5.67	0.013	2005
Weak IoT LCG	16	32.04 ± 11.21	0.500	46.12

*QRNG rates are classical-simulator rates on a laptop, not hardware rates.

the attacker recovered the seed and predicted the device's next key for 200 of 200 devices (100%), taking 1.39 ms on average per device on a laptop. No equivalent attack applies to the QRNG streams, which have no seed.

VI. DISCUSSION

Post-processing is required. A gate-based QRNG on today's NISQ hardware is not usable as-is: readout asymmetry alone produced a 2.2% overall bias that every NIST test detected and that an ML model exploited. Both extractors fixed this. Von Neumann is simple and exactly unbiased for independent bits but kept only 21.6% of the raw bits. SHA-256 kept 50% and also hides residual correlations, but it is a computational extractor rather than an information-theoretically proven one.

Per-qubit processing matters. Biases of different signs across qubits partly cancel when streams are pooled, so pooled tests can understate how biased individual qubits are. The Von Neumann corrector must also work on each qubit's own

stream, because pairing bits from two differently biased qubits does not remove bias. The NIST tests can therefore double as a per-qubit health check, as suggested in [3]: a device could exclude qubits like q5.

Statistical tests are necessary but not sufficient. The weak LCG passes the frequency-style proportions and has "perfect" entropy estimates, yet it is fully predictable, repeats keys, and its seed can be brute-forced in milliseconds. This matches the warning in NIST SP 800-22 itself that statistical testing cannot replace cryptanalysis [6]. It also explains why weak embedded generators keep turning up in deployed devices [1], [2].

Limitations. (1) This is a simulation study. A classical simulator draws its measurement outcomes from its own internal PRNG, so the "quantum" bits here are not truly quantum. What the study validates is the pipeline and the noise-handling behaviour. Certifying quantum origin requires running on real hardware (or a source-independent protocol [4]). (2) One calibration snapshot was used; real qubits drift over time. (3) Eight of the fifteen NIST tests were run. (4) Simulator throughput (about 1.98 Mbit/s) says nothing about cloud-hardware throughput, which is limited by queueing. For IoT use, the realistic deployment is a QRNG service that seeds a device's on-board CSPRNG, not a per-request call to a quantum computer.

VII. Conclusion and Future Work

We built and evaluated a simple, reproducible QRNG pipeline for IoT key generation. Under a realistic noise model based on *ibm_brisbane* calibration data, raw quantum bits failed all eight NIST tests and were partly predictable (52.02%). With per-qubit Von Neumann correction or SHA-256 extraction, the output passed all eight tests and was unpredictable to a learned model (50.06% and 49.81%). A weakly seeded IoT LCG, by contrast, was 95.03% predictable, repeated keys every 16 keys, and lost its seed in every trial. Future work will (i) run the same circuit on real IBM Quantum hardware and compare per-qubit bias with the calibration; (ii) replace SHA-256 with a seeded Toeplitz-hashing extractor and use NIST SP 800-90B min-entropy estimation; (iii) run the full fifteen-test NIST suite; and (iv) build an end-to-end demonstrator in which an ESP32 or Raspberry Pi fetches quantum seed material over TLS to seed its local CSPRNG.

References

[1] P. Kietzmann, T. C. Schmidt, and M. Wählisch, "A guideline on pseudorandom number generation (PRNG) in the IoT," *ACM Comput. Surv.*, vol. 54, no. 6, Art. 112, 2021.

[2] N. Heninger, Z. Durumeric, E. Wustrow, and J. A. Halderman, "Mining your Ps and Qs: Detection of widespread weak keys in network devices," in *Proc. 21st USENIX Security Symp.*, Bellevue, WA, 2012, pp. 205–220. [Online]. Available: https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/heninger

[3] K. Tamura and Y. Shikano, "Quantum random numbers generated by a cloud superconducting quantum computer," in *Int. Symp. Mathematics, Quantum Theory, and Cryptography*, Mathematics for Industry, vol. 33. Singapore: Springer, 2021, pp. 17–37.

[4] Y. Li, Y. Fei, W. Wang, X. Meng, H. Wang, Q. Duan, and Z. Ma, "Quantum random number generator using a cloud superconducting quantum computer based on source-independent protocol," *Sci. Rep.*, vol. 11, p. 23873, 2021, doi: 10.1038/s41598-021-03286-9. [Online]. Available: https://doi.org/10.1038/s41598-021-03286-9

[5] J. von Neumann, "Various techniques used in connection with random digits," *Nat. Bur. Stand. Appl. Math. Ser.*, vol. 12, pp. 36–38, 1951.

[6] A. Rukhin *et al.*, "A statistical test suite for random and pseudorandom number generators for cryptographic applications," NIST, Gaithersburg, MD, Special Publication 800-22 Rev. 1a, 2010.